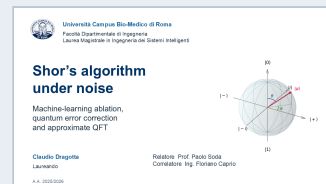


Discorso di laurea · 15 minuti

Leggi con calma, come se lo raccontassi. La barra azzurra con il triangolo indica quando cambiare slide; a destra vedi la slide che deve comparire. Durante la demo i tempi tra parentesi sono quelli del video.

► INIZIO · 1

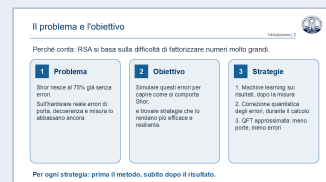
Copertina



Buongiorno a tutti. Ringrazio la commissione per la parola. Sono Claudio Dragotta e vi presento la mia tesi, che ho svolto con il professor Soda e l'ingegner Caprio.

► CAMBIA SLIDE · 2

Il problema e l'obiettivo



Ogni volta che pagate online o aprite la vostra banca dal telefono, i vostri dati sono protetti da un lucchetto matematico che si chiama RSA. Questo lucchetto regge su un'idea semplice: moltiplicare due numeri primi è facile, ma tornare indietro, per un numero di centinaia di cifre, è impossibile per qualunque computer di oggi.

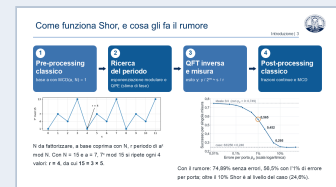
Nel 1994 Peter Shor ha dimostrato che un computer quantistico questo problema lo risolverebbe in modo efficiente.

C'è però un ma. Anche in un mondo perfetto, Shor non è infallibile: trova i fattori circa tre volte su quattro, il 75% dei casi. E i computer quantistici veri non sono perfetti. I qubit, le unità di informazione quantistica, sono fragilissimi: un minimo disturbo li altera. Le operazioni sbagliano, i qubit perdono l'informazione col passare del tempo, e anche la lettura finale può sbagliare.

La domanda della mia tesi è questa: cosa succede a Shor quando la macchina sbaglia, e che cosa posso fare per renderlo più resistente? Per rispondere ho simulato al computer questi errori e ho messo alla prova tre strategie.

► CAMBIA SLIDE · 3

Come funziona Shor, e cosa gli fa il rumore



Come funziona Shor? Prendiamo il mio caso: fattorizzare 15. Scelgo un numero di partenza, il 7, e calcolo le sue potenze, guardando solo il resto della divisione per 15. Esce 1, 7, 4, 13, e poi di nuovo 1, 7, 4, 13: la sequenza si ripete ogni quattro passi, come un ritmo. Quel 4 si chiama periodo, ed è tutto ciò che serve: da lì un semplice calcolo restituisce i fattori, 15 uguale 3 per 5.

Per numeri enormi questo ritmo è nascosto, e un computer classico non riesce a trovarlo. Il computer quantistico ci riesce perché esplora tutte le potenze insieme e fa emergere il ritmo grazie all'interferenza, come onde che si rinforzano. Nel mio caso il circuito usa 12 qubit.

Il grafico a destra mostra cosa succede quando la macchina sbaglia. Ho simulato il rumore così: dopo ogni operazione, con una piccola probabilità, il qubit subisce un errore casuale, si capovolge oppure perde il suo segno. In alto c'è il 75% ideale; in basso, tratteggiato, circa il 25%, cioè il risultato che otterrei tirando a caso.

Senza errori ottengo il 74,89%, esattamente come previsto. Ma basta un errore dell'1% per operazione, che sembra pochissimo, per scendere al 56%: il circuito contiene centinaia di operazioni e gli errori si sommano. Oltre il 10% Shor fa come chi tira a indovinare.

► CAMBIA SLIDE · 4

Obiettivi: dove interviene ciascuna strategia



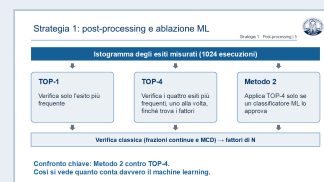
Per spiegare le tre strategie uso un'immagine. Pensate a una lettera che arriva rovinata dal viaggio. Avete tre possibilità: provare a interpretarla meglio dopo che è arrivata, proteggerla durante il viaggio, oppure scriverla più corta, così c'è meno da rovinare.

In giallo vedete i passi di Shor, in viola dove sono intervenuto. La prima lavora dopo la misura: provo a leggere meglio i risultati, anche con il machine learning. La seconda è la correzione degli errori, che protegge i qubit durante il calcolo; l'ho studiata in esperimenti dedicati, separati dal circuito di Shor. La terza è la QFT approssimata, che toglie operazioni al circuito.

Per ognuna vi mostro prima come funziona e subito dopo che cosa ho trovato.

► CAMBIA SLIDE · 5

Strategia 1: post-processing e ablazione ML



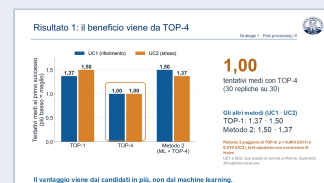
Prima strategia: leggere meglio i risultati. Un computer quantistico non dà una risposta sola. Eseguo il circuito 1024 volte e ottengo una classifica delle risposte, dalla più frequente alla meno frequente.

Ho confrontato tre modi. TOP-1 controlla solo la risposta più frequente. TOP-4 controlla, una alla volta, le prime quattro, finché trova i fattori. Il terzo, che nella tesi chiamo Metodo 2, aggiunge l'intelligenza artificiale: un classificatore decide se la classifica è affidabile, e solo allora si provano le prime quattro.

La domanda onesta è: se il Metodo 2 funziona, il merito è dell'intelligenza artificiale o dei tentativi in più? Per scoprirlo lo confronto con TOP-4, che fa gli stessi tentativi.

► CAMBIA SLIDE · 6

Risultato 1: il beneficio viene da TOP-4



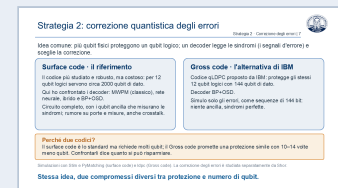
Ed ecco la risposta. Nel grafico, barre più basse significano meno tentativi per arrivare ai fattori. Nel riquadro arancione, TOP-4 trova i fattori sempre al primo colpo, in tutte le 30 ripetizioni dell'esperimento.

Il classificatore è molto accurato, eppure il Metodo 2 non migliora nulla, anzi fa un po' peggio, perché ogni tanto scarta proprio i risultati buoni. In pratica il classificatore premia le stesse risposte frequenti che TOP-4 controlla già: non aggiunge informazione. Aggiungo un limite: è stato addestrato su dati simulati, non su una macchina reale.

La lezione: uno strumento sofisticato va sempre confrontato con l'alternativa più semplice.

► CAMBIA SLIDE · 7

Strategia 2: correzione quantistica degli errori



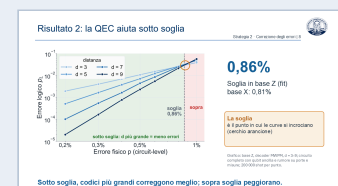
Seconda strategia: proteggere la lettera durante il viaggio. È la correzione quantistica degli errori. L'idea la conoscete: al telefono, se la linea è disturbata, ripetete la parola tre volte e vince la maggioranza.

Con i qubit c'è una difficoltà: non posso copiarli, e se li guardo direttamente distruggo l'informazione. Allora uso qubit di servizio, detti ancilla, che controllano se i qubit vicini sono d'accordo tra loro senza leggerli. I loro segnali si chiamano sindromi, e funzionano come campanelli d'allarme. Il decoder è il detective che, dai campanelli che suonano, capisce dov'è l'errore e come correggerlo.

Ho studiato due codici. Il surface code, il riferimento, è robusto ma costoso: per i 12 qubit di Shor ne servono circa 2000. L'ho simulato con il circuito completo, ancilla comprese, confrontando diversi decoder, tra cui una rete neurale. Il Gross code, proposto da IBM, promette una protezione simile con molti meno qubit; qui ho simulato solo gli errori, senza ancilla: un modello più semplice.

► CAMBIA SLIDE · 8

Risultato 2: la QEC aiuta sotto soglia



Il primo risultato è la soglia. Ogni colore è un codice con più qubit di protezione. Nella zona verde le curve dei codici più grandi stanno più in basso: aggiungere qubit riduce gli errori. Nella zona rossa succede il contrario: più qubit, più errori.

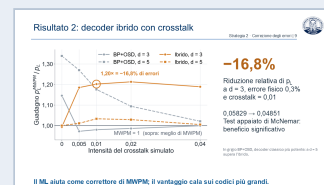
È come una squadra: aggiungere persone aiuta solo se ognuno sa già fare bene il proprio lavoro. Il punto in cui le curve si incrociano è la soglia, che ho stimato tra lo 0,8 e lo 0,9% di errore per operazione.

E per Shor cosa significa? Torniamo al grafico di prima. Se la correzione riduce l'errore che resta a 0,2% per operazione, Shor si mantiene intorno al 70%; a 0,5%, al 64%; senza correzione, all'1%, scende al 56%. La correzione aiuta, ma

non riporta Shor al 75%. È una stima: ho simulato Shor con un errore residuo, non l'intero circuito protetto.

► CAMBIA SLIDE · 9

Risultato 2: decoder ibrido con crosstalk



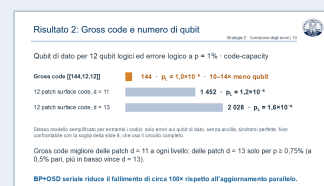
Qui l'intelligenza artificiale trova finalmente un ruolo. Nella realtà i qubit vicini si disturbano a vicenda, come conversazioni in una stanza affollata: si chiama crosstalk. Il decoder classico non tiene conto di questi errori collegati.

Una rete neurale da sola non batte il decoder classico. Ma usata come assistente che ne corregge le decisioni, il decoder ibrido, la linea arancione continua, funziona. Nel punto cerchiato gli errori scendono del 16,8%.

Sui codici più grandi, le linee tratteggiate, il vantaggio quasi sparisce, e un decoder classico più potente, BP+OSD, le linee grigie, fa ancora meglio. L'intelligenza artificiale aiuta da assistente, non da sostituto.

► CAMBIA SLIDE · 10

Risultato 2: Gross code e numero di qubit



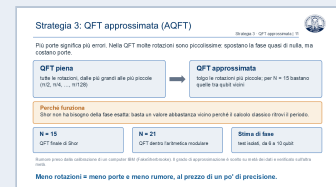
Infine il confronto tra i due codici. Per proteggere gli stessi 12 qubit, il Gross code, la barra arancione, usa 144 qubit; il surface code ne richiede da 1452 a 2028. Da 10 a 14 volte di meno.

Un dettaglio importante: per un confronto alla pari, qui anche il surface code è simulato nello stesso modello semplificato, senza ancilla. Per questo questi numeri non vanno confrontati con la soglia di prima, che usa il circuito completo.

E non protegge peggio: in questo modello, all'1% di errore sbaglia circa una volta su un milione, meglio delle soluzioni con il surface code. Anche qui il decoder conta: con la versione giusta gli errori calano di circa cento volte.

► CAMBIA SLIDE · 11

Strategia 3: QFT approssimata (AQFT)



Terza strategia: scrivere una lettera più corta. Ogni operazione è un'occasione di errore, quindi meno operazioni significa meno errori.

Ho lavorato sulla QFT, la trasformata di Fourier quantistica: la lente che trasforma il ritmo nascosto in picchi ben visibili. Al suo interno ci sono rotazioni sempre più piccole. Le ultime sono come girare una manopola di una frazione di grado: costano un'operazione intera, ma cambiano quasi nulla.

Nella versione approssimata le tolgo. Funziona perché Shor non ha bisogno della fase esatta: gli basta un valore abbastanza vicino perché il calcolo finale ritrovi il ritmo. L'ho provata su 15, su 21 e su test dedicati, con il rumore misurato su un vero computer IBM e riprodotto in simulazione.

► CAMBIA SLIDE · 12

Risultato 3: AQFT su N = 15



Ed ecco il risultato. A destra c'è la QFT completa, verso sinistra tolgo sempre più rotazioni. Con la QFT completa Shor si ferma al 48%. Con la versione scelta, quella cerchiata, sale al 63%: quasi 15 punti in più, con il 23% di operazioni a due qubit in meno.

Devo però essere onesto: 15 è un caso favorevole, perché il suo ritmo è di 4 e le rotazioni tolte non portavano informazione. Su 21 le operazioni si dimezzano, ma un vantaggio non è dimostrato. L'approssimazione va quindi scelta caso per caso.

► CAMBIA SLIDE · 13

Demo: Shor passo per passo, con e senza rumore



[Il video parte da solo e dura 2 minuti. Parla sopra il video, seguendo le scritte in basso; i tempi tra parentesi sono quelli del video.]

[0:00] Prima di concludere, vi faccio vedere il simulatore che ho realizzato. Segue Shor passo per passo, prima senza errori e poi con il rumore.

[0:06] È lo stesso caso: fattorizzare 15 partendo da 7. Ogni riga è un qubit; a destra, ogni sfera mostra con una freccia lo stato del suo qubit.

[0:17] Prima tutti i qubit vengono messi in sovrapposizione, cioè in più stati contemporaneamente. Poi le moltiplicazioni per 7 scrivono il ritmo nei qubit. Le frecce si accorciano, ma non è un errore: è l'entanglement, i qubit diventano legati tra loro.

[0:30] La QFT trasforma il ritmo in quattro picchi, e dalla misura un calcolo classico ricava 15 uguale 3 per 5.

[0:48] Ora lo stesso circuito con il rumore: le frecce si accorciano anche dove prima restavano intere. Lo stato si sta rovinando.

[1:08] Qui si vede dove entra il rumore: ogni operazione ha il suo errore, quelle su due qubit sono le più fragili, e anche la lettura finale può sbagliare.

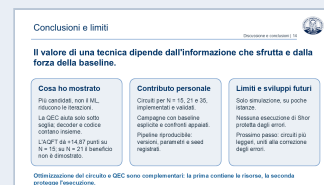
[1:15] Ripetendo il circuito molte volte, i risultati si sparpagliano su tutte le risposte possibili e i picchi si abbassano: è lo stesso calo che avete visto nel grafico all'inizio.

[1:33] Questa è informazione persa. Una volta misurata, non si recupera più.

[1:53] Ed è proprio il messaggio della mia tesi: il rumore va affrontato durante il calcolo, perché dopo la misura l'informazione distrutta non si può ricostruire.

► CAMBIA SLIDE · 14

Conclusioni e limiti



Torno alla lettera rovinata.

Interpretarla meglio dopo che è arrivata aiuta poco: nel post-processing il vantaggio viene dal provare più risposte, non dall'intelligenza artificiale.

Proteggerla durante il viaggio funziona, ma solo se la macchina è già abbastanza buona, sotto la soglia; e codice e decoder vanno scelti insieme. Accorciarla, con la QFT approssimata, aiuta, ma non in tutti i casi.

Il mio contributo è stato costruire e verificare i circuiti, progettare confronti onesti e rendere ogni risultato riproducibile.

I limiti li dichiaro: ho lavorato in simulazione, su pochi casi, e senza un'esecuzione di Shor completamente protetta. Il passo successivo è rendere i circuiti più leggeri e unirli alla correzione degli errori.

► CAMBIA SLIDE · 15

Grazie per l'attenzione



Vi ringrazio per l'attenzione. Sono a disposizione per le vostre domande.